



**Facultad de Ciencias Exactas y Naturales y Agrimensura –
UNNE**

TRABAJO PRÁCTICO INTEGRADOR - Grupo 44
Sistema de Semáforos Inteligentes y Análisis Comparativo de
Paradigmas

Paradigmas y lenguajes de programación - 2026

Introducción:

El presente informe tiene como objetivo exponer todo lo desarrollado, aprendido y profundizado durante la elaboración del trabajo práctico integrador presentado por los profesores de la asignatura de paradigmas y lenguajes, utilizando como base cada conocimiento desarrollado durante la cursada de la asignatura previamente mencionada.

Bitácora de Depuración: Fase 1

Requerimiento 2:

Causa: Error de lógica, de sintaxis y de estructuración.

Esta es una estructura que pensamos a priori para tratar de responder al requerimiento 2, tratamos de dividir todos los colores en funciones diferentes pero todos los caminos llevarían siempre a la misma condición, "rojo" si es que tratáramos de unir, puesto que al restar actual-tiempo por el valor recibido por parámetro siempre daría lo mismo: "0", ya que actual-tiempo toma el valor del parámetro recibido en la función.

Además al estructurar el sistema así, cada función individual asume que el tiempo "empieza de cero" para su propio color y termina colapsando porque las funciones no comparten una noción de tiempo acumulado. Es ahí en donde se nos ocurrió la forma de resolverlo.

```
1 (defun timer-rojo(tiempo-unix)
2   let ((actual-tiempo (tiempo-unix)))
3   (cond
4     ((<= (- actual-tiempo tiempo-unix) 90) 'rojo)
5     (t timer-amarillo(tiempo-unix))
6   )
7 )
8
9 (defun timer-amarillo(tiempo-unix)
10  let ((actual-tiempo (tiempo-unix)))
11  (cond
12    ((<= (- actual-tiempo tiempo-unix) 6) 'amarillo)
13    (t timer-verde(tiempo-unix))
14  )
15 )
16
17 (defun timer-verde(tiempo-unix)
18  let ((actual-tiempo (tiempo-unix)))
19  (cond
20    ((<= (- actual-tiempo tiempo-unix) 120) 'verde)
21    (t timer-rojo(tiempo-unix))
22  )
23 )
24
25 (defun timer (tiempo-unix)
26  (timer-rojo(tiempo-unix))
27 )
28
29 (print(timer 50))
```

Solución:

Necesitábamos que de una forma compartan un tiempo en común, un ciclo, por lo que se llegó al acuerdo de que si un ciclo de un semáforo (rojo - verde - amarillo) equivale a 216, $(90+120+6)$, deberíamos hacer que ese sea el número en común que compartan los 3. Es por ello que para saber en qué parte del ciclo va el semáforo, la mejor forma que obteniendo el resto de dividir el tiempo ingresado por parámetro por el total de lo que equivale un ciclo, 216, el cual siempre nos va a decir en qué parte se quedó ya que al ser una división de enteros, el resto siempre se mantendrá por debajo de 216.

```

6
7 (defun timer (tiempo-unix)
8   (let ((tiempo-ciclo (mod tiempo-unix 216)))
9     (cond
10      ((< tiempo-ciclo 90) 'rojo)
11      ((< tiempo-ciclo 210) 'verde)
12      (t 'amarillo))
13    )
14  )
15

```

Requerimiento 3:

Causa: Error de planificación y estructura

Esta fue la estructura pensada a priori para resolver el requerimiento 3, pero vimos que era mejor que de forma automática devuelva el historial de cambio de luces por lo que lo unificamos con el requerimiento 2.

```

1
2 ;req 3:
3
4 (defun log-transicion (tiempo-unix color-anterior color-nuevo)
5   (format t "Tiempo ~A: la luz ha cambiado de ~A a ~A~%"
6     tiempo-unix color-anterior color-nuevo))
7
8 (print(log-transicion 50 'desc 'rojo))
9

```

Solución:

Utilizamos una recursividad de cola para que vaya iterando cada vez que se va generando un cambio de estado dentro del ciclo de requerimiento 2 que lo unificamos dentro del requerimiento 3, y así va imprimiendo por pantalla el historial que pide la consigna por cada cambio de estado que vaya ocurriendo:

```

10 (defun timer (tiempo-unix)
11   (let ((tiempo-ciclo (mod tiempo-unix 216)))
12     (cond
13      ((< tiempo-ciclo 90) 'rojo)
14      ((< tiempo-ciclo 210) 'verde)
15      (t 'amarillo))
16    )
17  )
18
19 (defun log-transicion (tiempo-actual tiempo-fin color-anterior)
20   (let ((color-nuevo (timer tiempo-actual)))
21     (cond
22      ((= tiempo-actual tiempo-fin) 'NIL)
23      ((not (eq color-anterior color-nuevo))
24       (format t "Tiempo ~A: la luz ha cambiado de ~A a ~A~%"
25         tiempo-actual color-anterior color-nuevo)
26       (log-transicion (+ tiempo-actual 1) tiempo-fin color-nuevo)
27       (t (log-transicion (+ tiempo-actual 1) tiempo-fin color-anterior))))))
28

```

Requerimiento 4.a –

Problema :

Durante el desarrollo de la función duración-ciclo, surgió una dificultad para interpretar el requerimiento. No estaba claro si la duración que debía utilizarse correspondía al ciclo inicial ingresado por el usuario o a la duración de un ciclo ya calculado y almacenado.

Causa:

La descripción del requerimiento admitía más de una interpretación, lo que llevó a distintas propuestas de implementación.

Solución:

Luego de revisar los requerimientos, se decidió que la función debía calcular la duración total del ciclo a partir de la suma de los tiempos de las fases roja, verde y amarilla, garantizando que el valor obtenido representa la duración real del ciclo procesado.

Requerimiento 4.b –

Problema:

Al implementar la función recomendación-ciclo, surgió la duda de si utilizar estructuras if anidadas o la expresión cond para evaluar las distintas recomendaciones posibles.

Causa:

La función debía contemplar tres casos diferentes según la duración del ciclo, por lo que existían varias alternativas de implementación

```
(defun recomendacion-ciclo (duracion)
  (if (< duracion 35)
      "RECOMENDACION: Aumentar duracion del ciclo"
      (if (<= duracion 150)
          "RECOMENDACION: Mantener duracion"
          "RECOMENDACION: Reducir duracion del ciclo")))
```

Requerimiento 3 y 6:

Causa: Límite de memoria en entornos interpretados.

Al intentar ejecutar simulaciones largas en los requerimientos 3 y 6, la consola arrojaba *** - Program stack overflow. RESET. Esto pasaba porque los entornos que se nos dio en la cátedra como CLISP o XLISP-STAT leen el código “en crudo” y apilan cada llamada recursiva en la memoria física hasta agotarla antes de llegar al final de la simulación.

Requerimiento 3 -

```
[1]> (log-transicion 0 3000 'desconocido)
Tiempo 0: la luz ha cambiado de DESCONOCIDO a ROJO
Tiempo 90: la luz ha cambiado de ROJO a VERDE
Tiempo 210: la luz ha cambiado de VERDE a AMARILLO
Tiempo 216: la luz ha cambiado de AMARILLO a ROJO
Tiempo 306: la luz ha cambiado de ROJO a VERDE
Tiempo 426: la luz ha cambiado de VERDE a AMARILLO
Tiempo 432: la luz ha cambiado de AMARILLO a ROJO
Tiempo 522: la luz ha cambiado de ROJO a VERDE
Tiempo 642: la luz ha cambiado de VERDE a AMARILLO
Tiempo 648: la luz ha cambiado de AMARILLO a ROJO
Tiempo 738: la luz ha cambiado de ROJO a VERDE
Tiempo 858: la luz ha cambiado de VERDE a AMARILLO
Tiempo 864: la luz ha cambiado de AMARILLO a ROJO
Tiempo 954: la luz ha cambiado de ROJO a VERDE
Tiempo 1074: la luz ha cambiado de VERDE a AMARILLO
Tiempo 1080: la luz ha cambiado de AMARILLO a ROJO
Tiempo 1170: la luz ha cambiado de ROJO a VERDE
Tiempo 1290: la luz ha cambiado de VERDE a AMARILLO
Tiempo 1296: la luz ha cambiado de AMARILLO a ROJO
Tiempo 1386: la luz ha cambiado de ROJO a VERDE
Tiempo 1506: la luz ha cambiado de VERDE a AMARILLO
Tiempo 1512: la luz ha cambiado de AMARILLO a ROJO
Tiempo 1602: la luz ha cambiado de ROJO a VERDE
Tiempo 1722: la luz ha cambiado de VERDE a AMARILLO
Tiempo 1728: la luz ha cambiado de AMARILLO a ROJO
Tiempo 1818: la luz ha cambiado de ROJO a VERDE
Tiempo 1938: la luz ha cambiado de VERDE a AMARILLO
Tiempo 1944: la luz ha cambiado de AMARILLO a ROJO
Tiempo 2034: la luz ha cambiado de ROJO a VERDE
Tiempo 2154: la luz ha cambiado de VERDE a AMARILLO
Tiempo 2160: la luz ha cambiado de AMARILLO a ROJO
Tiempo 2250: la luz ha cambiado de ROJO a VERDE

*** - Program stack overflow. RESET
[2]> |
```

Requerimiento 6 -

```
[1]> (simular-distribucion 3600 'en-rojo 0 0 0 0 )

*** - Program stack overflow. RESET
```

Solución:

Forzamos la compilación de las funciones antes de ejecutarlas usando el comando (compile 'log-transicion) y (compile 'simular-distribucion). Esto permite que el sistema recicle el mismo bloque de memoria en cada iteración en lugar de apilar nuevos, evitando el desborde y permitiendo procesar miles de segundos sin problema.

Requerimiento 3 -

```
*** - Program stack overflow. RESET
[2]> (compile 'log-transicion)

LOG-TRANSICION ;
NIL ;
NIL
[3]> (log-transicion 0 3000 'desconocido)
Tiempo 0: la luz ha cambiado de DESCONOCIDO a ROJO
Tiempo 90: la luz ha cambiado de ROJO a VERDE
Tiempo 210: la luz ha cambiado de VERDE a AMARILLO
Tiempo 216: la luz ha cambiado de AMARILLO a ROJO
Tiempo 2586: la luz ha cambiado de VERDE a AMARILLO
Tiempo 2592: la luz ha cambiado de AMARILLO a ROJO
Tiempo 2682: la luz ha cambiado de ROJO a VERDE
Tiempo 2802: la luz ha cambiado de VERDE a AMARILLO
Tiempo 2808: la luz ha cambiado de AMARILLO a ROJO
Tiempo 2898: la luz ha cambiado de ROJO a VERDE
NO-HAY-CAMBIOS
[4]> |
```

Requerimiento 6 -

```
[2]> (compile 'simular-distribucion)

SIMULAR-DISTRIBUCION ;
NIL ;
NIL
[3]> (simular-distribucion 3600 'en-rojo 0 0 0 0 )

(ROJO 42.5 AMARILLO 2.6666667 VERDE 54.833336)
[4]> |
```

También nos dimos cuenta que esta misma función, en entornos modernos como SBCL, no nos arrojaba ningún error. Sin embargo, se optó por dejar esta instrucción para asegurar que cualquiera pueda ejecutar la simulación sin errores de memoria.

Autonomía y Ecosistema: Fase 2

El inicio del desarrollo de la fase 2 puso enfrente de nosotros nuevos retos para el trabajo, en este caso la inclusión de una librería al desarrollo de nuestros códigos.

Se nos presentó que debíamos asesorarnos sobre un gestor de bibliotecas para Common lisp llamado Quicklisp, el cual contiene más de 1500 bibliotecas, a los cuales se pueden acceder con unos comandos sencillos.

De esas 1500 bibliotecas 2 fueron las propuestas para que las utilizáramos en el proyecto: local-time: el cual si lo elegimos nos pide Modificar el Sistema de Auditoría (Requerimiento 3) para que, en lugar de imprimir el tiempo en formato Unix (Epoch), muestre la fecha y hora en un formato legible para humanos (Ej: [2026-05-16 14:30:00]).

cl-json: el cual nos pide Modificar el sistema para que los tiempos de temporización por color (90s, 6s, 120s) no estén fijos en el código, sino que se carguen dinámicamente desde un archivo .json de configuración externa.

El grupo debatió y llegó a la conclusión que haríamos uso de la biblioteca local-time porque nos pareció la más eficaz de realizar y la más elegante por el formato que usa para imprimir por pantalla.

Decidido eso, empezamos a investigar sobre como incorporar la librería al núcleo de nuestra aplicación, desde la misma página de quicklisp descargamos el gestor que contiene la librería y de paso nos informamos que debíamos descargar el entorno, compilador y motor de ejecución moderno SBCL(Steel Bank Common Lisp) para poder probar la librería puesto que para que Quicklisp pueda ejecutarse, descargar bibliotecas y usarlas, necesita del entorno y compilador que entienda e interprete sus instrucciones.

Una vez ya descargado todo e instalado tanto el entorno como el gestor, procedimos a llamar a la librería de quicklisp que utilizaremos: Local-time y la colocamos al inicio de todo el programa.

```
1 (load (merge-pathnames "quicklisp/setup.lisp" (user-homedir-pathname))) ;ruta directa universal a quicklisp
2
3 (ql:quickload "local-time" :silent t) ; cargamos la libreria
4
5 (sb-ext:unlock-package :sb-ext) ; para poder usar timer como nombre ya que es una palabra reservada
6
7 (defun timer (tiempo-unix)
8   (let ((tiempo-ciclo (mod tiempo-unix 216)))
9     (cond
10      ((< tiempo-ciclo 90) 'rojo)
11      ((< tiempo-ciclo 210) 'verde)
12      (t 'amarillo))
13   )
14 )
15
```

La línea 1 es una ruta directa a la ubicación de la descarga quicklisp por si no reconoce el ingreso manual de la misma.

En la línea 3 invocamos a la librería localtime y hacemos uso de la palabra reservada silent para evitar que aparezcan mensajes molestos propios de la librería.

Luego procedemos a poner en prueba la librería con el siguiente caso de prueba:

(log-transición 1778893800 1778894300 'desconocido)

```
Tiempo [2026-5-16 1:10:0]: la luz ha cambiado de DESCONOCIDO a VERDE
Tiempo [2026-5-16 1:11:54]: la luz ha cambiado de VERDE a AMARILLO
Tiempo [2026-5-16 1:12:0]: la luz ha cambiado de AMARILLO a ROJO
Tiempo [2026-5-16 1:13:30]: la luz ha cambiado de ROJO a VERDE
Tiempo [2026-5-16 1:15:30]: la luz ha cambiado de VERDE a AMARILLO
Tiempo [2026-5-16 1:15:36]: la luz ha cambiado de AMARILLO a ROJO
Tiempo [2026-5-16 1:17:6]: la luz ha cambiado de ROJO a VERDE

-----
(program exited with code: 0)

Presione una tecla para continuar . . .
```

Cómo se puede observar, las variables en tiempo unix - epoch se cambiaron a formato humano donde se observa una elegante presentación y de muy alta legibilidad.

Estudio Comparativo : Fase 3

Clojure es un lenguaje de programación funcional moderno perteneciente a la familia de los lenguajes Lisp. Fue creado por Rich Hickey en 2007 y se caracteriza por ejecutarse principalmente sobre la Máquina Virtual de Java (JVM), aunque también existen implementaciones para JavaScript y .NET. Su diseño está orientado a la programación funcional, la inmutabilidad y el manejo eficiente de la concurrencia.

Actualmente, Clojure se utiliza en distintas áreas de la industria, entre ellas el desarrollo web, el procesamiento y análisis de datos, la inteligencia artificial y las aplicaciones empresariales. Algunas empresas reconocidas que utilizan Clojure en parte de sus desarrollos son Netflix, Walmart, Nubank, Atlassian y Cisco.

1. Ambos lenguajes son Lisps, pero Clojure elimina los símbolos cotizados (:rojo) en favor de los Keywords (:rojo). ¿Cuál es la ventaja técnica y de rendimiento de un Keyword en Clojure para mapear estados?

En Clojure, los Keywords, como por ejemplo :rojo, :verde o :amarillo, son objetos inmutables y autoevaluables, por lo que no es necesario utilizar comillas para evitar su evaluación. Además, Clojure mantiene una única instancia de cada Keyword en memoria, lo que permite realizar comparaciones y búsquedas de manera más eficiente.

Esta característica resulta especialmente útil para representar estados en el trabajo que realizamos, ya que los Keywords pueden utilizarse como claves de mapas, permitiendo asociar fácilmente información a cada estado. Por ejemplo, es posible almacenar la duración correspondiente a cada color del semáforo mediante una estructura de datos simple y eficiente. Gracias a ello, el código es más legible y se obtiene un mejor rendimiento en las operaciones.

2. Clojure prohíbe la mutabilidad por defecto usando estructuras de datos persistentes. ¿Cómo maneja Clojure el paso del tiempo en el simulador semafórico sin modificar variables?

Clojure se basa en el principio de inmutabilidad, por lo que las estructuras de datos no se modifican directamente. En lugar de cambiar el valor de una variable existente, cada transformación produce una nueva versión del estado, conservando las versiones anteriores.

En el caso nuestro, el paso del tiempo puede representarse mediante una sucesión de estados. Cada vez que cambia el color del semáforo o se actualiza el tiempo restante, se genera un nuevo estado que contiene los valores actualizados, mientras que el estado anterior permanece igual. De esta forma, la evolución temporal del sistema se modela como una secuencia de estados inmutables.

Este enfoque reduce la posibilidad de errores provocados por modificaciones accidentales de datos, facilita el seguimiento del comportamiento del sistema y mejora el manejo de procesos concurrentes, una de las principales ventajas de la programación funcional y del lenguaje Clojure.

Conclusión:

La realización de este trabajo nos permitió poner en práctica los conocimientos adquiridos durante el cursado de la materia y enfrentarnos a situaciones reales de diseño, implementación y depuración. A medida que avanzamos en cada fase, fuimos comprendiendo mejor las características de la programación funcional y la importancia de las distintas herramientas disponibles para facilitar el desarrollo. Además, el análisis comparativo con Clojure nos permitió ampliar nuestra visión sobre los lenguajes de la familia Lisp y sus aplicaciones actuales en un rango general. En definitiva, este proyecto nos ayudó a entender mejor conceptos, desarrollar una mejor capacidad de análisis y adquirir experiencia práctica valiosa utilizando herramientas como Github Lisp y Clojure simulando nuestra entrada en el mundo laboral, situación que nos va a dar mayores herramientas de razonamiento y nos prepara mejor para nuestro futuro profesional.

Integrantes del grupo:

- Medrano Ignacio Nahuel
- Rodriguez Martemucci Lucciana Angelinna
- Juan Esteban Martinez
- Isler Mendiaz Magno Exequiel
- Villagra Facundo Nahuel

Tema Investigado:

- Cómo usar e integrar librerías de quicklisp.
- Clojure: usos, sintaxis y aplicaciones.

Bibliografía:

- https://www-sbcl-org.translate.goog/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc
- <https://www.quicklisp.org/beta/>
- <https://lispcookbook.github.io/cl-cookbook/getting-started.html>
- <https://www.youtube.com/watch?v=inhLMMBB77c&t=8>
- <https://building.nubank.com/es/programacion-funcional-con-clojure-por-que-y-como-nubank-la-usa-y-escala-tan-bien/>
- <https://clojure.org/about/lisp>
- <https://es.wikipedia.org/wiki/Clojure>
- <https://stackoverflow.com/questions/15269193/stack-overflow-from-recursive-function-call-in-lisp>
- https://novaspec.org/cl/21_1_Stream_Concepts